AIMScribe — Integration Guide for CMED

What CMED needs to build, and what CMED needs to send.

AIMS LAB · Independent University, Bangladesh · 10 September 2026

---

1. The whole integration in one page

AIMScribe records the conversation between a doctor and a patient, so that it can later be transcribed and turned into a draft prescription. It cannot know *which* consultation it is recording. Only CMED knows that.

So CMED tells it. Three signals, across two channels.

	Signal	Sent when	Sent to
API 1	Trigger	The doctor opens the patient's details	The recorder on the doctor's PC
API 2	Patient information	Immediately after the recording starts	The AIMS LAB backend
API 3	Prescription built	The doctor builds the prescription	Both — see §5

That is the entire integration. Everything else in this document explains those three signals.

What CMED does not have to do. No audio to capture, store or handle. No cryptographic keys to hold or rotate. No changes to the CMED database. No new login system. No software to install on the doctor's PC. No screen to build — every message the doctor sees comes from our own small on-screen control, not from CMED.

---

2. The two channels

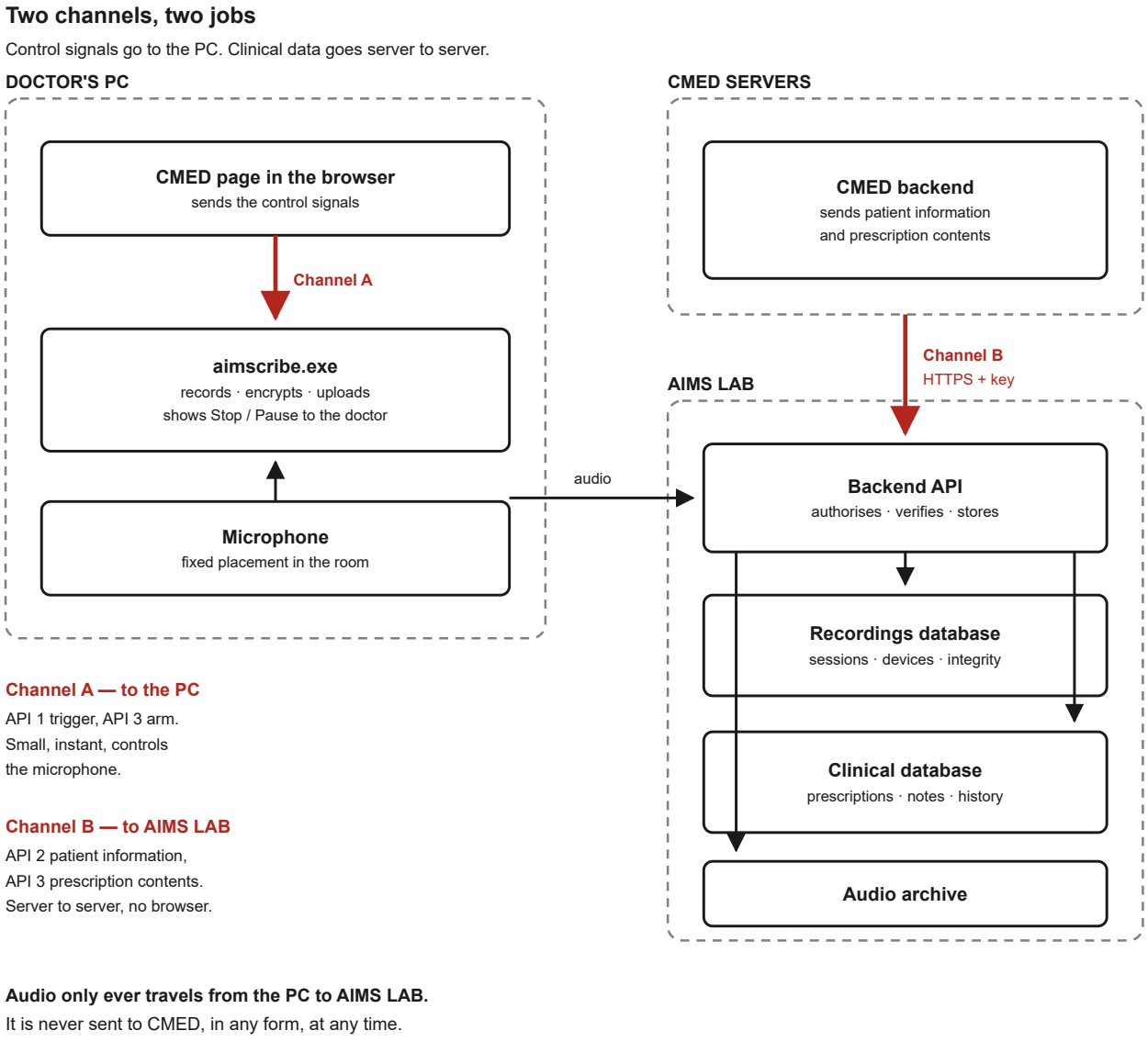

Figure 1 | Two channels, two jobs. Control signals travel to the recorder on the doctor's own PC. Clinical data travels server to server from CMED to the AIMS LAB backend. Audio travels only from the PC to AIMS LAB, and never to CMED.

Channel A — CMED page to the recorder on the same PC. Address: `ws://127.0.0.1:5050/ws`. This is a connection from the CMED page in the doctor's browser to a program running on that same computer. It never leaves the machine. It carries only small control messages, because it has to be instant — the microphone must open the moment the doctor opens the patient.

Why a local connection and not a call to our server? Because the recorder is on the doctor's PC. A message that had to travel to a server and back before the microphone opened would lose the first seconds of the consultation, which is exactly when the patient says why they came.

Channel B — CMED backend to the AIMS LAB backend. Address: `https://<aims-backend>/api/v2/clinical/...`, authenticated with an API key we issue to CMED. This carries the clinical data: patient information, prescriptions, notes, history. It is server to server, so payload size does not matter and nothing sensitive passes through a browser.

---

3. One consultation, start to finish

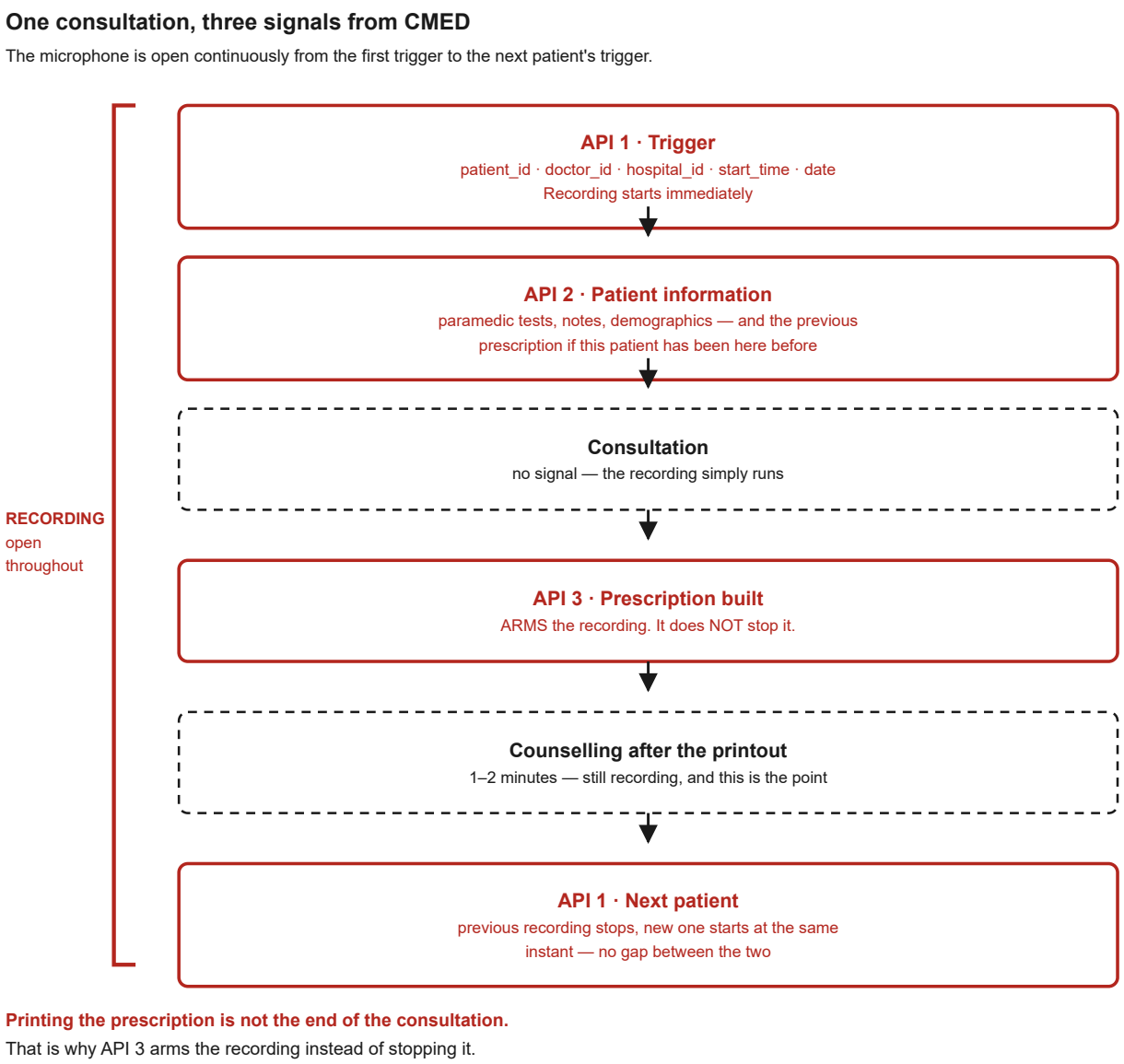

Figure 2 | The three signals across one consultation. The recording starts on API 1 and stays open continuously until the next patient's API 1. API 3 arms the recording; it does not stop it.

The point most easily missed is API 3. Printing the prescription is not the end of the consultation. The doctor prints, hands the paper over, and then counsels the patient for another minute or two — and

that counselling is exactly the part worth recording. So API 3 does not stop the recording. It *arms* it, meaning: this consultation is finished, so the next patient is now allowed to end it.

The recording actually stops when the **next** patient's API 1 arrives. At that moment the previous recording closes and the new one opens at the same instant, on a separate thread, so there is no gap between the two patients.

---

4. API 1 — Trigger (start the recording)

When: the doctor opens the patient's details or history. **Where:** Channel A, the local connection.

```
{
  "command": "start",
  "request_id": "cmed-7f3a2b91",
  "trigger": {
    "patient_id":  "P0012345",
    "doctor_id":   "DR0042",
    "hospital_id": "CMED-DHK-BANANI-01",
    "start_time":  "2026-09-10T10:14:32+06:00",
    "date":        "2026-09-10"
  }
}
```

Field	Required	Rule
`patient_id`	Yes	Letters, numbers, `_` and `-` only. Up to 64 characters. Must be the same value every time for the same patient.
`doctor_id`	Yes	Same rule. Never leave it blank — we will refuse rather than guess.
`hospital_id`	Yes	Same rule. Must be the clinic this PC belongs to.
`start_time`	Yes	Full date and time with the time zone, e.g. `+06:00`.
`date`	Yes	`YYYY-MM-DD`.
`request_id`	Helpful	Anything you like; we send it back so you can match the reply.

Why `patient_id` must be stable. It becomes the folder and file name for that patient's recordings. If it changes for the same patient, their recordings become two unrelated sets and cannot be put back together afterwards.

Reply:

```
{ "event": "ack", "command": "start", "request_id": "cmed-7f3a2b91",
  "status": 200, "code": "RECORDING_STARTED",
  "data": { "session_id": "01JB8XQ4M7YZ2K9V3N5P6R8T0W" } }
```

Keep `session_id`. It identifies this consultation, and you send it back with API 2 and API 3.

---

5. API 2 — Patient information (right after the recording starts)

When: as soon as the recording has started. **Where:** Channel B, server to server.

Before the patient enters the doctor's room, a paramedic records their details and takes basic measurements. The doctor needs those at the start of the consultation, and so do we. If the patient has been to this hospital before, we also need their most recent prescription — the doctor refers to it, and our system needs it to make sense of what is said.

```
POST /api/v2/clinical/patient-information
X-CMED-Key: <the key we issue to you>
```

```
{
  "session_id":  "01JB8XQ4M7YZ2K9V3N5P6R8T0W",
  "patient_id":  "P0012345",
  "demographics": {
    "name": "…", "sex": "female", "age_years": 34,
    "phone": "…", "address": "…"
  },
  "paramedic": {
    "recorded_at": "2026-09-10T10:06:00+06:00",
    "weight_kg": 58.0, "height_cm": 156,
    "blood_pressure": "120/80", "pulse_bpm": 78,
    "temperature_c": 37.1, "spo2_percent": 98,
    "notes": "…"
  },
  "previous_visit": {
    "date": "2026-06-02",
    "prescription": { "…": "…" },
    "diagnoses": ["…"],
    "notes": "…"
  }
}
```

`previous_visit` is `null` for a patient who has not been to this hospital before. Send everything you hold for a returning patient — we would rather receive a field we do not yet use than discover later that it was never sent.

The fields required differ between male and female patients. Send what applies; our database enforces the right rules for each.

---

6. API 3 — Prescription built (arm the recording)

This one does two things at once, on both channels.

6a. Arm the recording — Channel A, to the PC.

```
{ "command": "prescription_built",
  "request_id": "cmed-7f3a2b92",
  "patient_id": "P0012345",
  "session_id": "01JB8XQ4M7YZ2K9V3N5P6R8T0W",
  "occurred_at": "2026-09-10T10:26:11+06:00" }
```

Reply: `200 GATE_ARMED`. **The recording keeps running.**

6b. Send the prescription — Channel B, server to server.

```
POST /api/v2/clinical/prescription
X-CMED-Key: <your key>
```

```
{
  "session_id":  "01JB8XQ4M7YZ2K9V3N5P6R8T0W",
  "patient_id":  "P0012345",
  "issued_at":   "2026-09-10T10:26:11+06:00",
  "diagnoses":   ["…"],
  "items": [
    { "drug": "…", "dose": "…", "frequency": "…", "duration": "…",
      "instructions": "…" }
  ],
  "investigations": ["…"],
  "advice": "…",
  "follow_up": "2026-10-10",
  "notes": "…"
}
```

Send `items` as a list, one entry per prescribed medicine, rather than as one block of text. We store each field in its own column, and a list we have to take apart later loses information that was there when you sent it.

---

7. What happens when the next patient arrives

Nothing new to send. You simply send **API 1 again**, for the new patient.

Situation	What we do
API 3 was received for the current patient	Previous recording closes, new one opens at the same instant. No gap.
API 3 was **not** received	We refuse the new trigger with `409 GATE_NOT_ARMED`, and the current recording keeps running

That second row is deliberate. If a doctor clicks another patient by accident in the middle of a consultation, the current recording is not cut in half. It keeps going. Your page can ignore the `409` — nothing is broken, and nothing needs to be shown to the doctor.

---

8. What comes back

Branch on `code`, never on `message`. The message text is shown to doctors and will be reworded; the code will not change.

`status`	`code`	Meaning	What CMED should do
200	`RECORDING_STARTED`	Recording	Keep the `session_id`
200	`GATE_ARMED`	API 3 accepted	Nothing
400	`MISSING_FIELD`	A required field is absent	Fix the payload — a build-time bug
400	`INVALID_IDENTIFIER`	An id has characters we do not allow	Fix the identifier
401	`AUTHORISATION_FAILED`	We could not authorise this recording	Log it; do not retry automatically
401	`CLINIC_MISMATCH`	This PC does not belong to that hospital	Log and tell us; the mapping is wrong
403	`ORIGIN_NOT_ALLOWED`	Your page address is not on our list	Configuration — tell us the address
409	`GATE_NOT_ARMED`	A consultation is still open	**Expected.** Do nothing.
423	`DEVICE_NOT_ENROLLED`	This PC is not registered	Tell us which machine
503	`AGENT_NOT_READY`	Recorder is starting	Retry once after a moment
—	*no connection*	Recorder not installed or not running	Log it and carry on

The rule that matters more than any of the above: if anything at all goes wrong with AIMScribe, your page must carry on as if it were not there. A lost recording is a lost recording. A doctor who cannot see a patient because a recorder failed is a much worse outcome. In practice this is one line:

```
aimscribe.startConsultation({ … }).catch(() => {});
```

---

9. How we know the request is genuine

CMED does not hold any key and does not have to prove anything. The checks are ours, and they run on our side. This section is here so your security reviewer can see what they are.

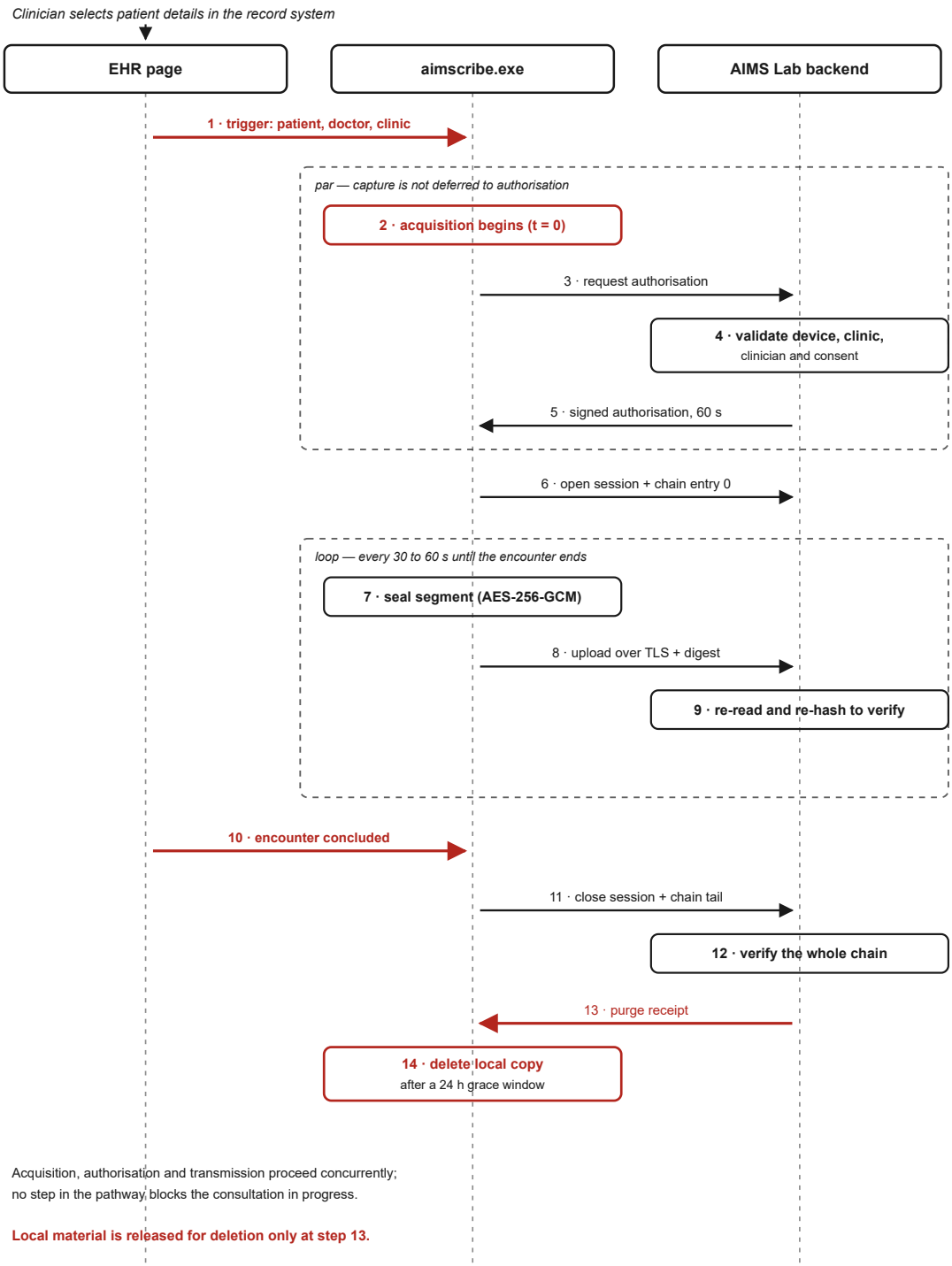

Figure 3 | What happens behind the trigger. The recording starts immediately,

while authorisation is checked in parallel, so a slow network never costs the opening of the consultation.

Every PC is registered once, before it is used. An administrator from AIMS LAB issues a one-time activation code for that specific machine. The first time the recorder runs, it exchanges that code for a permanent identity and generates a private key that never leaves the computer. The code is single-use and expires. After that, the machine is known to us and is tied to one clinic.

A registered PC still cannot record just because someone asks it to. Every consultation needs its own short-lived permission, which the recorder requests from our backend at the moment of the trigger. Our backend checks that the doctor exists and works at that clinic, that the clinic matches the machine's registration, and that consent is recorded. The permission it issues lasts sixty seconds and works once.

Why both. Registration answers *may this machine record at all*. The per-consultation permission answers *is this particular request genuine*. Without the second, any web page the doctor happened to open could start a recording on a registered machine. With it, even someone who obtained CMED access could not start or intercept a recording.

One thing we need from you for this to work: the exact web address your page is served from — scheme, host and port, for both production and testing. We put it on an allowlist, and a connection from anywhere else is refused before it is even accepted.

---

10. What never crosses between us

Direction	Never
AIMS LAB → CMED	Audio, in any form. Transcripts. Any AI-generated output through this interface.
CMED → AIMS LAB	Anything you have not agreed to send. We ask for nothing beyond §5 and §6.

Audio is recorded on the doctor's PC, encrypted before it is written to disk, uploaded to AIMS LAB, and deleted from the PC as soon as we confirm we hold a verified copy. It never travels to CMED.

---

11. What we need from you to begin

#	What	Effort
1	The exact page addresses, production and testing	10 minutes
2	Your clinic identifiers, and which AIMS LAB hospital each maps to	30 minutes, once
3	Confirmation that `patient_id` never changes for a patient	—
4	Open the local connection from your page and keep it open	half a day
5	Send API 1 when the doctor opens patient details	half a day
6	Send API 3 when the prescription is built	quarter of a day
7	Send API 2 and the prescription from your backend	one day
8	Handle the replies by `code` ; ignore failures quietly	half a day
9	Joint testing with us	one day
	Total	**3–4 developer-days**

We provide a single test page that connects to the recorder, sends each signal and prints the reply, so your developer can see the whole thing working before writing any CMED code.

---

12. Who to ask

Send questions to AIMS LAB, Independent University Bangladesh. If something in this document is ambiguous, it is a fault in the document — tell us and we will fix it rather than leave you to guess.